Point-by-point response to the reviews of COLA-D-26-00245

This document answers each point of the two reports received on manuscript COLA-D-26-00245 (*Journal of Computer Languages*, decision: reject). The paper was substantially revised and most experiments were re-run; every number below is generated from the stored result files, as in the manuscript. Section numbers refer to the revised manuscript. Where a point could not be fully addressed, we say so.

Reviewer 1

Comment. The abstract is too long and hard to read; the English, style and organisation prevented the reviewer from understanding the message; terms such as "lexical tokens", "engineered operation taxonomy" and "the parser's own grammar node kinds" were not understandable.

Response. We rewrote the abstract (about 200 words, one idea per sentence, no unexplained terms) and the introduction. The introduction now defines the four ways of describing a node with one running example (`av_malloc(n)`), before any result is stated, and the same example is used in Section 3.1; Table 2 defines every node description precisely. The introduction states what changed in our conclusion after review and ends with a paragraph on the organisation of the paper. Sections were reorganised around five numbered research questions, each opening with a one-sentence plain summary. We also removed the claim that "abandoning lexical features proved useful" in the earlier framing, which the reviewer rightly found confusing: the revised paper says exactly what each comparison does and does not show.

Reviewer 2

1. **Paper organisation at the end of Section 1.** Done: the last paragraph of Section 1 ("Organisation").

2. **Tables 1, 4 and 9 are not cited.** Every table, figure and algorithm is now referenced in the text; we verified this mechanically against the `\labels`. Table numbering also changed with the revision.

3. **No statistical analysis/testing.** Section 4.5 (statistical treatment): every paired comparison now reports the mean per-fold difference, a cluster bootstrap over held-out projects (95% interval and p; a project is one cluster however many folds it appears in), an exact two-sided Wilcoxon signed-rank test over the ten folds and a second one over five repeat-averaged blocks (the two repeats of the split are not independent, see point 9), and Holm-adjusted p-values for each family of contrasts. The bootstrap is the primary inference; the fold-level Wilcoxon tests do not survive

Holm adjustment over a family of this size, which the manuscript states. A chi-square test is not appropriate for our data (paired scores, not counts); for the one count-based question, whether truncation depends on the label, we use Fisher's exact test.

4. **Are the datasets balanced? How handled?** No: prevalence is 0.059 (BigVul) and 0.058 (DiverseVul). Section 4.2 describes the handling: class-weighted training loss, untouched test sets, threshold-free metrics (AUC, AUPRC) and the trivial all-positive floor.

5. **Block diagram.** Figure 1 (Section 3.1).

6. **Algorithm/pseudocode.** Algorithms 1 (graph and node views) and 2 (cross-project evaluation), Section 3.6.

7. **Ablation study.** Section 4.11 (RQ4) and Table 13: edge-type removal and, in Section 4.8, removal of text, anonymisation, kind and operation type individually, plus capacity-matched controls. The thirty-two-type granularity run was not repeated and the claim based on it was withdrawn.

8. **Comparison with related work.** Section 4.12 (RQ5): re-implementations of the input representation of Devign/ReVeal (text plus node type), anonymised sequences, a Transformer over the node sequence, and a pretrained UniXcoder encoder (frozen, linear probe), on the same samples and folds; plus a function-size rule. These are comparisons of input representations inside one protocol, not reproductions of the published systems, and the manuscript says so.

9. **Fold counts do not match; headline rests on 5 folds; the 32-vs-8 comparison used one fold and a 5-fold run.** Fixed, with a disclosed limitation. Every condition of every table now runs on the same ten folds, and the analysis script refuses to merge result files with differing fold assignments. The ten folds are two repeats of a five-way project-level split; because projects are dealt out largest first, the largest projects fall into the same fold index in both repeats, so the folds are better described as five folds with partial replication. The manuscript says so (Section 4.4), the bootstrap treats each project as one cluster, and a Wilcoxon test over the five repeat-averaged blocks is reported. The thirty-two-type condition was not re-run and the claim is dropped; the sixteen-type comparison uses 10 matched folds (Section 4.11).

10. **Lexical baseline underspecified; add lexical+kind and anonymised+kind.** Section 3.5 and Table 2 specify what leaves and internal nodes carry in each view. The lexical view gives every internal node one `<INTERNAL>` token, so it contains no node-type information; unseen tokens map to `<UNK>`, and the dictionary is built from the training projects of each fold only (the earlier draft built it from all projects, contrary to what the text said; that is fixed). We added lexical+κ (the Devign-style input) and anonymised+κ and anonymised-only views, and positioned the work against prior node-type features (Section 2.1). This changed a conclusion: renaming identifiers is about as good as using grammar kinds (Section 4.8).

11. **κ+ϕ never compared directly with κ.** Table 10 reports the paired contrast on both corpora: BigVul -0.004 [-0.017, +0.021], DiverseVul +0.008 [-0.005, +0.022] AUC (GGNN). The DiverseVul point estimate the reviewer noticed was an artefact of comparing two separate gaps.

12. **Architecture claim too broad.** Narrowed (Section 4.10): the manuscript now states that the evidence covers small networks trained from scratch. We added a Transformer over the node sequence (BigVul) and a frozen pretrained UniXcoder probe on both corpora. We did *not* fine-tune LineVul or UniXcoder, because the machine used for the experiments has no GPU;

this is stated as a limitation. The pretrained probe is competitive with the graph views, which is also reported.

13. **Small, non-random samples; Android dominates; prevalence 0.054 vs 8.8%.** Samples are now uniform random (fixed seed) draws of 10,000 functions per corpus from the whole training split of the public mirrors, giving 555 and 567 vulnerable functions (Section 4.2); we did not go larger because the grid already costs more than a day of CPU time. Results are repeated without the dominant test project (`Android`, up to 64% of a test fold) and per project (Table 9). The difference between overall and test prevalence is explained in Section 4.2: the two largest repositories, which have lower prevalence, are held train-only.

14. **Validation corpora differ from experimental corpora; who annotated; second annotator; thin classes.** Section 4.7 (RQ1) was redone. Callee names are now drawn from the experimental samples of BigVul and DiverseVul as well as Devign (212, 227 and 236 names). Two independent blind annotators labelled every name; agreement is Cohen's $\kappa = 0.91$ and disagreements were resolved by a third blind pass. **Important caveat:** all three annotators are language models; no human annotator was available for this revision. The original 212 Devign labels were assigned by a single, non-blind annotator and agree with the new consensus at $\kappa = 0.87$. Per-class counts and Wilson intervals are reported, and the smallest class has 20 positives, so thin classes remain thin. A human second annotator would strengthen this further. Macro precision against the consensus is 0.90, lower than the 0.976 of the first submission, which we now regard as optimistic.

15. **Missing hyper-parameters; effect of the 200-node cap.** Table 5 gives hidden size, learning rate, epochs, batch size, loss weighting, dictionary size, node cap and seeds; there is no early stopping and no validation split, and the seed is 1000+fold. The cap truncates 18.1% of BigVul and 25.6% of DiverseVul functions, and it affects the vulnerable class much more: 42.3% of vulnerable against 16.6% of benign functions in BigVul (Fisher $p < 0.001$). Vulnerable functions are simply longer, which led us to add a function-size baseline that, notably, outperforms every network (Section 4.8, Section 4.12).

Changes that go beyond the reviewers' requests

- The dictionary leakage noted under point 10 was found while specifying the lexical baseline and corrected.
- The "capacity control" reported in the first submission had no code or data behind it; it was replaced by an actual experiment.
- The first submission's RQ1 labels were produced non-blind; they were replaced as described under point 14.
- The central claim was reframed. Grammar kinds are no longer presented as better than anonymised text, and the manuscript states plainly that all learned models trail a function-size rule.